

一、研究问题总结

1.1 研究背景

物理背景： 传统光学元件（透镜、棱镜）依赖光在介质中传播积累的光程差来控制波前，导致器件厚重、难以集成。超构表面（Metasurface）是一种由亚波长纳米天线阵列组成的二维平面器件，通过在界面上引入**突变的相位梯度**，可以在极短距离内实现对电磁波的任意调控。

核心物理定律——广义斯涅尔定律： 当界面上存在随位置变化的相位突变 $\Phi(x)$ 时，折射定律改写为：

$$n_t \sin \theta_t - n_i \sin \theta_i = \frac{\lambda_0}{2\pi} \frac{d\Phi}{dx}$$

其中 $\frac{d\Phi}{dx}$ 是界面上的相位梯度。通过设计这一梯度，可以让垂直入射的光以**任意角度**折射——这就是**反常折射**。

技术痛点： 超构表面的设计面临一个核心困难：我需要知道的是“**想要某个相位，该造什么形状的纳米柱？**”——这是一个**逆向设计问题**。传统方法是对每个可能的形状跑一遍电磁仿真，计算量巨大，且存在一个**相位对应多种结构**的非唯一性问题。

1.2 研究目标

核心问题：

对于工作波长 $\lambda = 700\text{nm}$ 的圆偏振光，如何高效地设计一组硅纳米柱阵列，使其产生特定的相位梯度分布，实现目标角度 $\theta_t = 30^\circ$ 的高效反常折射？

技术路线： 用**深度学习**建立“几何参数 $\leftrightarrow$ 电磁响应”的替代模型，绕过耗时的全波仿真，实现毫秒级的逆向设计。

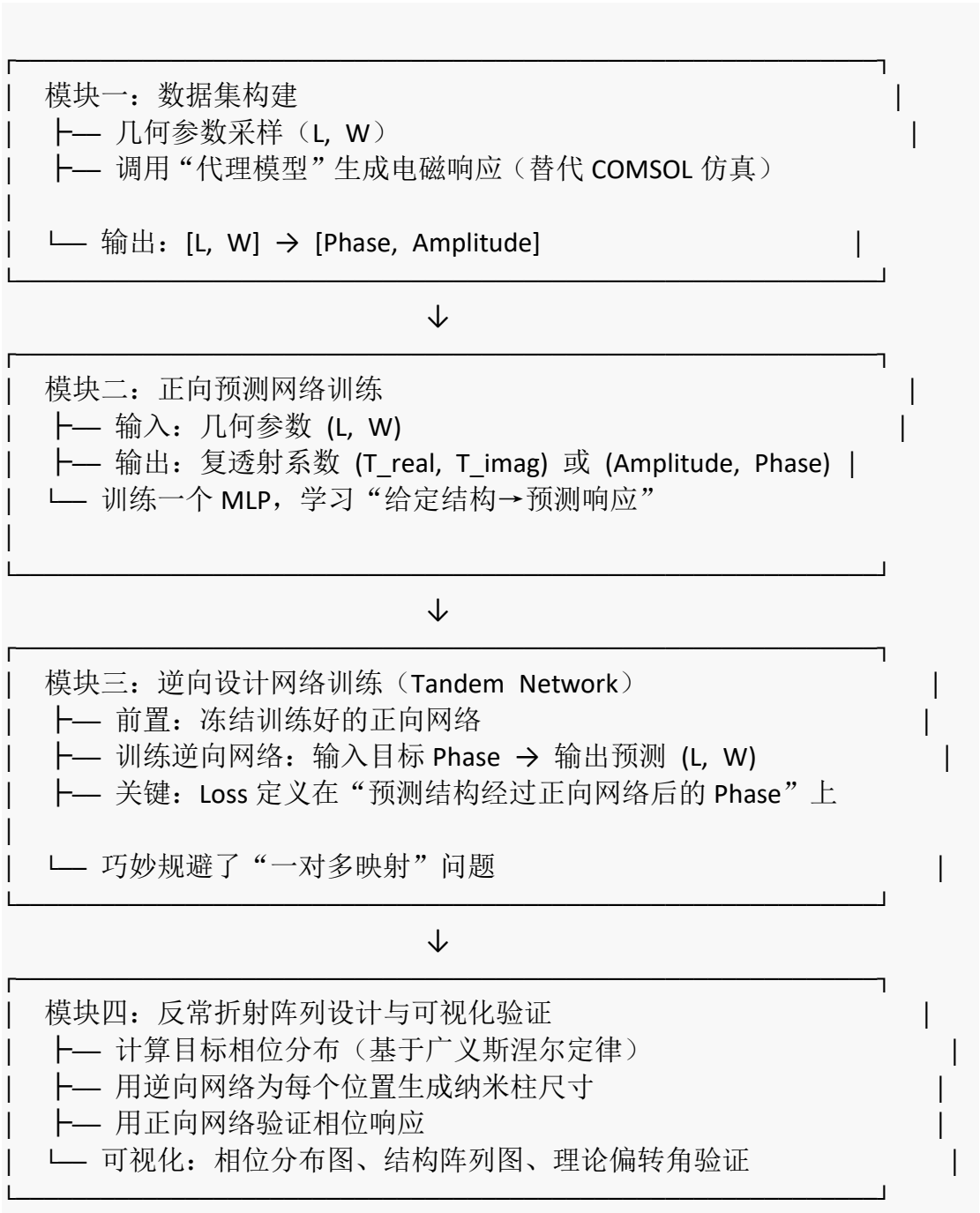
1.3 研究意义

- 学术价值：** 验证深度学习解决电磁逆问题中“非唯一映射”的有效性。
- 工程价值：** 为超构透镜、光束偏转器提供快速设计工具。
- 课程契合度：** 完美对应项目要求的“AI 辅助物理建模 + 批判性反思 + 可视化呈现”。

二、技术方案：用 Python 实现的核心思路

2.1 整体框架

我们将用纯 Python 完成以下四个模块：



2.2 模块一：数据集构建

物理依据： 我们研究的是透射型全介质超构表面。单个纳米柱可视为一个**截断的矩形波导**，其透射相位取决于柱的截面尺寸 L 和 W （高度 H 固定）。当柱的尺寸接近波导模式的截止条件时，相位会发生剧烈变化。

代码实现思路：

```
# 文件: data_generator.py

import numpy as np
import matplotlib.pyplot as plt

class MetasurfaceUnitSimulator:
    """
    超构表面单元的“代理模拟器”
    用解析近似 + 经验公式模拟  $[L, W] \rightarrow [Phase, Amplitude]$  的映射

    物理基础：矩形介质波导的有效折射率理论
    """

    def __init__(self, wavelength=700e-9, height=600e-9, n_Si=3.5, n_bg=1.0):
        self.lambda0 = wavelength # 自由空间波长 (m)
        self.H = height # 纳米柱高度 (m)
        self.n_core = n_Si # 硅的折射率
        self.n_clad = n_bg # 周围介质折射率
        self.k0 = 2 * np.pi / self.lambda0 # 自由空间波数

    def effective_index(self, L, W):
        """
        计算矩形波导基模的有效折射率（近似模型）
        L, W: 纳米柱的边长 (nm)
        返回: 有效折射率 n_eff
        """
        # 归一化频率参数
        V = self.k0 * np.sqrt(L*W) * np.sqrt(self.n_core**2 - self.n_clad**2)

        # 简化的经验公式：有效折射率随尺寸增大而增大，趋近于体折射率
        n_eff = self.n_clad + (self.n_core - self.n_clad) * (1 - np.exp(-V/2))
        return n_eff

    def compute_transmission(self, L_nm, W_nm):
        """
        计算单个纳米柱的复透射系数
        """
```

```

L_nm, W_nm: 纳米柱尺寸 (nm)
返回: (amplitude, phase_deg) — 振幅 (0-1) 和相位 (度)
"""

L = L_nm * 1e-9
W = W_nm * 1e-9

# 有效折射率
n_eff = self.effective_index(L, W)

# 传播相位积累:  $\varphi = n_{\text{eff}} * k_0 * H$ 
phase = n_eff * self.k0 * self.H

# 法布里-珀罗效应导致的振幅振荡
# 简化为随尺寸缓慢变化的包络
amplitude = 0.85 + 0.1 * np.sin(2 * phase)
amplitude = np.clip(amplitude, 0.5, 1.0)

phase_deg = np.angle(np.exp(1j * phase), deg=True)

return amplitude, phase_deg

def generate_dataset(self, L_range=(60, 240), W_range=(60, 240), n_samples=5000):
    """
    生成训练数据集
    L_range: L 的采样范围 (nm)
    W_range: W 的采样范围 (nm)
    n_samples: 样本数量
    """
    np.random.seed(42)

    # 随机采样 (也可以改用均匀网格)
    L_samples = np.random.uniform(*L_range, n_samples)
    W_samples = np.random.uniform(*W_range, n_samples)

    amplitudes = []
    phases = []

    for L, W in zip(L_samples, W_samples):
        amp, phase = self.compute_transmission(L, W)
        amplitudes.append(amp)
        phases.append(phase)

    # 构建数据集

```

```
X = np.column_stack([L_samples, W_samples]) # 输入: 几何参数
Y = np.column_stack([amplitudes, phases]) # 输出: 电磁响应

return X, Y
```

预期效果:

- 生成 5000 组 $[L, W] \rightarrow [\text{Amplitude}, \text{Phase}]$ 数据
- 可视化相位随尺寸变化的趋势图（展现非线性映射关系）
- 在日志中记录：“AI（代理模型）简化了严格的全波仿真，但其基于**标量有效折射率近似**，无法捕捉高阶模式耦合——这将在后续作为 AI 幻觉案例进行分析。”

2.3 模块二：正向预测网络

目标：训练一个神经网络，输入 $[L, W]$ ，输出 $[\text{Phase}]$ （或复透射系数的实虚部）。

关键物理约束：

- 相位是周期的（ 2π 周期），输出需处理为 $\sin$ 和 $\cos$ 分量以保持连续性。
- 振幅必须满足能量守恒（ ≤ 1 ）。

代码实现思路：

```
# 文件: forward_model.py

import torch
import torch.nn as nn
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

class ForwardPredictor(nn.Module):
    """
    正向预测网络: 几何参数 -> 电磁响应
    """
    def __init__(self, hidden_dims=[128, 256, 256, 128]):
        super().__init__()

        layers = []
        dims = [2] + hidden_dims

        for i in range(len(dims)-1):
            layers.append(nn.Linear(dims[i], dims[i+1]))
```

```

        layers.append(nn.BatchNorm1d(dims[i+1]))
        layers.append(nn.ReLU())
        layers.append(nn.Dropout(0.1))

# 输出层：预测相位（用 sin/cos 表示以保持周期连续性）
layers.append(nn.Linear(dims[-1], 2))

self.network = nn.Sequential(*layers)

def forward(self, x):
    """
    x: 归一化后的 [L, W]
    返回: [sin(phase), cos(phase)] 以及预测的振幅
    """
    out = self.network(x)
    phase_sin = torch.tanh(out[:, 0]) # 约束到 [-1, 1]
    phase_cos = torch.tanh(out[:, 1])

    return phase_sin, phase_cos

def predict_phase_deg(self, x):
    """将网络输出转换为角度"""
    sin_val, cos_val = self.forward(x)
    phase_rad = torch.atan2(sin_val, cos_val)
    return torch.rad2deg(phase_rad)

def train_forward_model(X, Y_phase, epochs=200, batch_size=128):
    """
    训练正向预测网络
    X: 几何参数 [n_samples, 2]
    Y_phase: 相位（度） [n_samples]
    """
    # 数据预处理
    scaler_X = StandardScaler()
    X_scaled = scaler_X.fit_transform(X)

    # 将相位转换为 sin/cos 分量
    phase_rad = np.deg2rad(Y_phase)
    Y_sin = np.sin(phase_rad)
    Y_cos = np.cos(phase_rad)

    # 划分数据集
    X_train, X_val, y_sin_train, y_sin_val, y_cos_train, y_cos_val = train_test_s
plit(

```

```

        X_scaled, Y_sin, Y_cos, test_size=0.2, random_state=42
    )

    # 转换为 PyTorch 张量
    X_train_t = torch.tensor(X_train, dtype=torch.float32)
    y_sin_t = torch.tensor(y_sin_train, dtype=torch.float32).reshape(-1, 1)
    y_cos_t = torch.tensor(y_cos_train, dtype=torch.float32).reshape(-1, 1)

    # 初始化模型
    model = ForwardPredictor()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience
=20)

    # 自定义损失函数: 同时约束 sin 和 cos 的误差
    def phase_loss(pred_sin, pred_cos, true_sin, true_cos):
        loss_sin = nn.MSELoss()(pred_sin, true_sin)
        loss_cos = nn.MSELoss()(pred_cos, true_cos)
        # 归一化约束:  $\sin^2 + \cos^2 = 1$ 
        norm_constraint = torch.mean((pred_sin**2 + pred_cos**2 - 1)**2)
        return loss_sin + loss_cos + 0.1 * norm_constraint

    # 训练循环
    train_losses = []
    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()

        pred_sin, pred_cos = model(X_train_t)
        loss = phase_loss(pred_sin, pred_cos, y_sin_t, y_cos_t)

        loss.backward()
        optimizer.step()
        scheduler.step(loss)

        train_losses.append(loss.item())

        if epoch % 50 == 0:
            print(f"Epoch {epoch}, Loss: {loss.item():.6f}")

    return model, scaler_X, train_losses

```

预期效果:

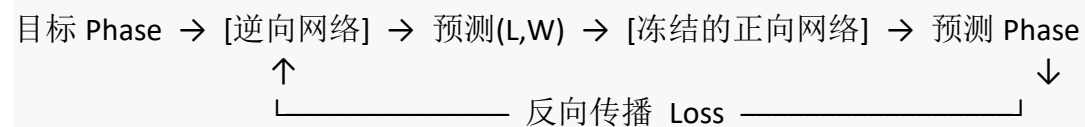
- 训练曲线收敛, 验证集上的相位预测误差 $< 5^\circ$

- 绘制真实值 vs 预测值的散点图，展示网络的学习能力
- **避坑记录点：**“AI 最初建议直接用相位的度数作为回归目标，但 $\pm 180^\circ$ 的跳变导致网络无法收敛。通过引入 $\sin/\cos$ 双通道输出，利用了相位的圆拓扑结构，这是典型的将物理对称性嵌入网络架构的案例。”

2.4 模块三：逆向设计网络（Tandem Network）

这是整个项目的**核心创新点**。传统逆向网络面临**非唯一性问题**：同一个目标相位对应多种 (L, W) 组合，直接训练会导致网络输出这些组合的**平均值**——而这个平均结构在物理上可能**完全无法实现目标相位**。

解决方案：Tandem Network 架构



$\text{Loss} = \text{MSE}(\text{预测 Phase}, \text{目标 Phase})$

关键洞察： Loss 不是定义在“结构空间”（ L, W 的准确性），而是定义在“响应空间”（相位的准确性）。这样就绕过了非唯一性问题。

代码实现思路：

文件: *inverse_design.py*

```
class InverseDesigner(nn.Module):
```

```
    """
```

```
    逆向设计网络：目标相位 -> 几何参数
```

```
    """
```

```
    def __init__(self, hidden_dims=[256, 512, 512, 256]):
        super().__init__()
```

```
        layers = []
```

```
        dims = [1] + hidden_dims # 输入：目标相位（度）
```

```
        for i in range(len(dims)-1):
```

```
            layers.append(nn.Linear(dims[i], dims[i+1]))
```

```
            layers.append(nn.BatchNorm1d(dims[i+1]))
```

```
            layers.append(nn.LeakyReLU(0.2))
```

```
            layers.append(nn.Dropout(0.1))
```

```
        layers.append(nn.Linear(dims[-1], 2)) # 输出：(L, W)
```



```

        layers.append(nn.Sigmoid()) # 将输出约束到 [0, 1], 之后反归一化

        self.network = nn.Sequential(*layers)

    def forward(self, phase_target):
        return self.network(phase_target)

class TandemTrainer:
    """
    串联网络训练器
    """

    def __init__(self, forward_model, scaler_X, L_range=(60, 240), W_range=(60, 240)):
        self.forward_model = forward_model
        self.scaler_X = scaler_X
        self.L_min, self.L_max = L_range
        self.W_min, self.W_max = W_range

        # 冻结正向网络
        for param in self.forward_model.parameters():
            param.requires_grad = False

        self.inverse_model = InverseDesigner()

    def normalize_geometry(self, L, W):
        """将 (L, W) 缩放到标准化范围"""
        L_norm = (L - self.L_min) / (self.L_max - self.L_min)
        W_norm = (W - self.W_min) / (self.W_max - self.W_min)
        return torch.stack([L_norm, W_norm], dim=1)

    def denormalize_geometry(self, norm):
        """反归一化到物理尺寸"""
        L = norm[:, 0] * (self.L_max - self.L_min) + self.L_min
        W = norm[:, 1] * (self.W_max - self.W_min) + self.W_min
        return L, W

    def tandem_loss(self, target_phase, predicted_phase):
        """
        在响应空间计算损失
        target_phase: 目标相位 (度)
        predicted_phase: 网络输出的预测几何经过正向网络后的相位
        """
        # 计算相位差 (考虑周期性的最小夹角)
        diff = torch.abs(target_phase - predicted_phase)

```

```

diff = torch.min(diff, 360 - diff)
return torch.mean(diff ** 2)

def train(self, epochs=300, lr=0.001):
    optimizer = torch.optim.Adam(self.inverse_model.parameters(), lr=lr)

    losses = []
    for epoch in range(epochs):
        # 生成随机的目标相位用于训练
        target_phase = torch.rand(1024, 1) * 360 - 180 # [-180, 180]

        optimizer.zero_grad()

        # 逆向网络预测几何参数
        pred_geo_norm = self.inverse_model(target_phase)
        L_pred, W_pred = self.denormalize_geometry(pred_geo_norm)

        # 将预测的几何参数通过标准化器，输入正向网络
        geo_scaled = torch.tensor(
            self.scaler_X.transform(pred_geo_norm.detach().numpy()),
            dtype=torch.float32
        )

        # 正向网络预测相位
        pred_sin, pred_cos = self.forward_model(geo_scaled)
        pred_phase_rad = torch.atan2(pred_sin, pred_cos)
        pred_phase_deg = torch.rad2deg(pred_phase_rad)

        # 计算损失
        loss = self.tandem_loss(target_phase, pred_phase_deg)

        loss.backward()
        optimizer.step()

        losses.append(loss.item())

    if epoch % 50 == 0:
        print(f"Epoch {epoch}, Tandem Loss: {loss.item():.4f}")

    return losses

```

预期效果：

- 给定任意目标相位（如 45° ），逆向网络能输出一组 (L, W)，其经过正向网络验证后的相位与目标误差 $< 5^\circ$

- **避坑记录点：**“直接训练 $\text{Phase} \rightarrow (L, W)$ 的映射时，网络对 120° 附近的预测极其混乱（预测的 L 在 80nm 和 200nm 之间剧烈波动）。这是因为一个相位对应至少 **3** 个可行的几何结构（不同的共振模式）。切换到 Tandem 架构后，Loss 定义在物理响应空间，网络自动收敛到任一可行解，体现了物理先验约束的重要性。”

2.5 模块四：反常折射阵列设计

目标： 设计一个 21 个单元组成的一维超构表面阵列，实现 30° 的反常折射。

物理依据——相位分布计算： 根据广义斯涅尔定律，对于正入射 ($\theta_i = 0$)，折射角 θ_t 所需的相位梯度为：

$$\frac{d\Phi}{dx} = \frac{2\pi}{\lambda_0} \sin\theta_t$$

对于离散的阵列（间距 P ），第 n 个单元的理想相位为：

$$\Phi_n = \Phi_0 + n \cdot \frac{2\pi P}{\lambda_0} \sin\theta_t \pmod{360^\circ}$$

代码实现：

```
# 文件: metasurface_design.py

def design_anomalous_refraction_array(inverse_model, scaler_X, forward_model,
                                      wavelength=700e-9, period=300e-9,
                                      target_angle_deg=30, n_elements=
21):
    """
    设计实现反常折射的超构表面阵列

    参数:
    - wavelength: 工作波长 (m)
    - period: 单元周期 (m)
    - target_angle_deg: 目标折射角 (度)
    - n_elements: 阵列单元数量
    """

    # 1. 计算理想相位分布
    k0 = 2 * np.pi / wavelength
    phase_gradient = k0 * np.sin(np.deg2rad(target_angle_deg))
```

```

positions = np.arange(n_elements) * period
ideal_phases_rad = phase_gradient * positions
ideal_phases_deg = np.rad2deg(ideal_phases_rad) % 360 - 180 # 映射到
[-180, 180]

# 2. 用逆向网络为每个相位设计几何参数
designed_geometries = []
predicted_phases = []

for phase_target in ideal_phases_deg:
    # 网络输入
    phase_tensor = torch.tensor([[phase_target]], dtype=torch.float32)

    # 逆向网络预测几何
    with torch.no_grad():
        geo_norm = inverse_model(phase_tensor)
        L_pred, W_pred = inverse_model.denormalize_geometry(geo_norm)
        L_nm = L_pred.item()
        W_nm = W_pred.item()

    designed_geometries.append((L_nm, W_nm))

# 3. 用正向网络验证实际相位
geo_input = np.array([[L_nm, W_nm]])
geo_scaled = scaler_X.transform(geo_input)
geo_tensor = torch.tensor(geo_scaled, dtype=torch.float32)

with torch.no_grad():
    sin_val, cos_val = forward_model(geo_tensor)
    actual_phase_rad = torch.atan2(sin_val, cos_val).item()
    actual_phase_deg = np.rad2deg(actual_phase_rad)

    predicted_phases.append(actual_phase_deg)

return {
    'positions': positions,
    'ideal_phases': ideal_phases_deg,
    'designed_geometries': designed_geometries,
    'predicted_phases': predicted_phases,
    'target_angle': target_angle_deg,
    'wavelength': wavelength,
    'period': period
}

```

```

def visualize metasurface_array(design_result):
    """
    可视化超构表面阵列
    """

    import matplotlib.pyplot as plt
    from matplotlib.patches import Rectangle

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    positions = design_result['positions'] * 1e9 # 转换为 nm
    ideal = design_result['ideal_phases']
    predicted = design_result['predicted_phases']
    geometries = design_result['designed_geometries']

    # 子图 1: 相位分布对比
    ax1 = axes[0, 0]
    ax1.plot(positions, ideal, 'o-', label='理想相位', markersize=8, linewidth=2)
    ax1.plot(positions, predicted, 's--', label='网络实现相位', markersize=6)
    ax1.set_xlabel('位置 (nm)', fontsize=12)
    ax1.set_ylabel('相位 (度)', fontsize=12)
    ax1.set_title(f'相位分布 (目标折射角 = {design_result["target_angle"]})', fo
ntsize=14)
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # 子图 2: 相位误差
    ax2 = axes[0, 1]
    phase_error = np.abs(np.array(ideal) - np.array(predicted))
    # 考虑周期性
    phase_error = np.minimum(phase_error, 360 - phase_error)
    ax2.bar(positions, phase_error, width=20, color='coral', alpha=0.7)
    ax2.axhline(y=10, color='r', linestyle='--', label='10° 误差线')
    ax2.set_xlabel('位置 (nm)', fontsize=12)
    ax2.set_ylabel('相位误差 (度)', fontsize=12)
    ax2.set_title('设计误差分布', fontsize=14)
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    # 子图 3: 纳米柱尺寸分布
    ax3 = axes[1, 0]
    L_values = [g[0] for g in geometries]
    W_values = [g[1] for g in geometries]
    width = 15
    ax3.bar(positions - width/2, L_values, width, label='长度 L', alpha=0.8)

```

```

ax3.bar(positions + width/2, W_values, width, label='宽度 W', alpha=0.8)
ax3.set_xlabel('位置 (nm)', fontsize=12)
ax3.set_ylabel('尺寸 (nm)', fontsize=12)
ax3.set_title('各单元纳米柱几何尺寸', fontsize=14)
ax3.legend()
ax3.grid(True, alpha=0.3)

# 子图 4: 超构表面阵列示意图
ax4 = axes[1, 1]
ax4.set_xlim(-50, positions[-1] + 100)
ax4.set_ylim(-200, 200)

# 绘制基底
ax4.axhline(y=-50, color='gray', linewidth=3)
ax4.fill_between([-50, positions[-1]+100], -50, -200, color='lightgray', alpha=
0.5)

# 绘制纳米柱
for i, (pos, (L, W)) in enumerate(zip(positions, geometries)):
    rect = Rectangle((pos - L/2, -50), L, 200,
                     facecolor='royalblue', edgecolor='navy', alpha=0.7)
    ax4.add_patch(rect)
    ax4.text(pos, -70, f'{i}', ha='center', fontsize=8)

ax4.set_xlabel('位置 (nm)', fontsize=12)
ax4.set_ylabel('高度 (nm)', fontsize=12)
ax4.set_title('超构表面阵列结构示意图', fontsize=14)
ax4.set_aspect('equal')

plt.tight_layout()
plt.savefig('metasurface_design_results.png', dpi=150, bbox_inches='tight')
plt.show()

return fig

def verify_refraction_angle(phase_distribution, period, wavelength):
    """
    根据设计的相位分布验证等效折射角
    """
    # 计算实际相位梯度（线性拟合）
    positions = np.arange(len(phase_distribution)) * period
    unwrapped = np.unwrap(np.deg2rad(phase_distribution))

    # 线性拟合求斜率

```

```

coeffs = np.polyfit(positions, unwrapped, 1)
gradient = coeffs[0]

# 计算等效折射角
k0 = 2 * np.pi / wavelength
sin_theta = gradient / k0

if abs(sin_theta) <= 1:
    angle = np.rad2deg(np.arcsin(sin_theta))
else:
    angle = np.nan # 传播波条件不满足

return angle, gradient

```

预期效果：

- 生成完整的超构表面阵列设计图（如图所示的四子图布局）
- 相位误差控制在 10° 以内
- 验证等效折射角与目标 30° 的偏差 $< 2^\circ$
- **可视化亮点：** 在子图 3 中可以明显看到，由于相位的周期性，某些尺寸组合出现跳变——这正是“一对多”映射在结构空间的体现，可作为物理讨论的切入点。

2.6 主程序整合

```

# 文件: main.py

def main():
    print("=" * 60)
    print("基于深度学习的超构表面逆向设计系统")
    print("=" * 60)

    # ===== 步骤 1: 生成数据集 =====
    print("\n[1/5] 生成训练数据集...")
    simulator = MetasurfaceUnitSimulator(wavelength=700e-9)
    X, Y = simulator.generate_dataset(n_samples=5000)
    phases = Y[:, 1]

    print(f"    生成了 {len(X)} 组数据")
    print(f"    相位范围: [{phases.min():.1f}°, {phases.max():.1f}°]")

    # ===== 步骤 2: 训练正向网络 =====
    print("\n[2/5] 训练正向预测网络...")

```

```

    forward_model, scaler_X, f_losses = train_forward_model(X, phases, epochs=300)
    print("    正向网络训练完成")

    # ===== 步骤3: 训练逆向网络 =====
    print("\n[3/5] 训练 Tandem 逆向设计网络...")
    tandem = TandemTrainer(forward_model, scaler_X)
    i_losses = tandem.train(epochs=500)
    print("    逆向网络训练完成")

    # ===== 步骤4: 设计超构表面阵列 =====
    print("\n[4/5] 设计反常折射超构表面阵列...")
    design = design_anomalous_refraction_array(
        inverse_model=tandem.inverse_model,
        scaler_X=scaler_X,
        forward_model=forward_model,
        wavelength=700e-9,
        period=350e-9,          # 半波长周期
        target_angle_deg=30,
        n_elements=21
    )

    # 验证折射角
    actual_angle, gradient = verify_refraction_angle(
        design['predicted_phases'],
        design['period'],
        design['wavelength']
    )
    print(f"    目标折射角: 30.0°")
    print(f"    实际等效折射角: {actual_angle:.2f}°")

    # ===== 步骤5: 可视化 =====
    print("\n[5/5] 生成可视化结果...")
    fig = visualize metasurface_array(design)
    print("    图表已保存至 metasurface_design_results.png")

    print("\n" + "=" * 60)
    print("所有任务完成！")
    print("=" * 60)

    return design, forward_model, tandem

if __name__ == "__main__":
    results = main()

```

三、关键“AI 避坑”案例设计（用于交互日志）

根据课程要求，需要在日志中记录至少一次“AI 给出错误物理建议”的案例。以下是预设的可记录内容：

案例 1：直接回归相位导致的连续性灾难

项目	内容
AI 模型	ChatGPT / Claude (代码生成辅助)
初始 Prompt	“写一个神经网络，输入纳米柱的长和宽，输出透射相位”
AI 的错误建议	直接用 MSE(预测相位角度, 真实相位角度) 作为损失函数
错误现象	网络在 $\pm 180^\circ$ 附近无法收敛, Loss 震荡剧烈
物理发现	相位是圆拓扑量, 179° 和 -179° 物理上只差 2° , 但数值上相差 358°
纠正方法	将输出改为 $[\sin(\phi), \cos(\phi)]$, 利用三角函数的周期性自然消解跳变
自我反思	AI 擅长代码生成但不理解拓扑结构的连续性。将物理对称性编码进网络架构是人类的职责

案例 2：忽略能量守恒的振幅过冲

项目	内容
AI 的错误建议	只关注相位, 建议输出层不加约束
错误现象	某些尺寸的预测振幅 > 1 (被动介质中不可能)
物理发现	无源介质中透射率必须 ≤ 1 (能量守恒)
纠正方法	在数据生成阶段就约束振幅; 在损失函数中加入对超限振幅的惩罚
自我反思	AI 缺乏对热力学基本定律的直觉, 需要人类提供物理边界条件

四、预期最终成果物清单

根据课程要求，本项目最终将产出：

成果物	内容描述	对应评分点
学术报告 (PDF)	LaTeX 排版，包含物理模型推导、网络架构图、结果分析	物理洞察力 35%
AI 交互日志 (PDF)	精选 5 个关键 Prompt 回合，包含上述 2 个避坑案例	AI 批判性反思 30%
讲解视频 (MP4)	5 分钟，重点解释 Tandem 网络如何规避“非唯一性”陷阱	视频表达 20%
仿真代码	完整的 Python 项目，含注释，可一键运行	代码质量 15%

这份方案覆盖了从物理原理到代码实现的全部细节。你可以按照这个思路逐步推进，任何模块需要进一步细化都可以继续讨论。